```c
// 1) To Implement Merge sortusing Divide & Conquer approach for the
user entered input and then find the running time of implementation
using time() function

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int arr[], int l, int m, int r)
{
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
    {
        L[i] = arr[l + i];
    }
    for (int j = 0; j < n2; j++)
    {
        R[j] = arr[m + 1 + j];
    }
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k++] = L[i++];
        }
        else
        {
            arr[k++] = R[j++];
        }
    }
    while (i < n1)
    {
        arr[k++] = L[i++];
    }
    while (j < n2)
    {
        arr[k++] = R[j++];
    }
}
```

```c
void mergeSort(int arr[], int l, int r)
{
    if (l < r)
    {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main()
{
    int n;
    printf("Enter number of elements you want to enter: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d Array Elements: ", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    printf("Original Array: ");
    printArray(arr, n);
    clock_t start, end;
    start = clock();
    mergeSort(arr, 0, n - 1);
    end = clock();
    printf("Sorted Array: ");
    printArray(arr, n);
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Running Time = %f seconds", time_taken);
    return 0;
}
```

```c
// 2) To Implement Quick sort using Divide & Conquer approach for the
user entered input and then find the running time of implementation
using time() function.

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

void printArray(int arr[], int n)
{
```

```
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main()
{
    int n;
    printf("Enter number of elements you want to enter: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d Array Elements: ", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    printf("Original Array: ");
    printArray(arr, n);
    clock_t start, end;
    start = clock();
    quickSort(arr, 0, n - 1);
    end = clock();
    printf("Sorted Array: ");
    printArray(arr, n);
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Running Time = %f seconds", time_taken);
    return 0;
}
```

```
// 3) To Implement Matrix Chain Multiplication using Dynamic
Programming approach for the user entered input and then find the
running time of implementation using time() function.

#include <stdio.h>
#include <limits.h>
#include <time.h>

int matrixChain(int p[], int n)
{
```

```
    int m[n][n];
    for (int i = 1; i < n; i++)
    {
        m[i][i] = 0;
    }
    for (int L = 2; L < n; L++)
    {
        for (int i = 1; i < n - L + 1; i++)
        {
            int j = i + L - 1;
            m[i][j] = INT_MAX;
            for (int k = i; k < j; k++)
            {
                int cost = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] *
p[j];
                if (cost < m[i][j])
                {
                    m[i][j] = cost;
                }
            }
        }
    }
    return m[1][n - 1];
}

int main()
{
    int n;
    printf("Enter number of elements you want: ");
    scanf("%d", &n);
    int p[n + 1];
    printf("Enter %d Array elements: ", n + 1);
    for (int i = 0; i <= n; i++)
    {
        scanf("%d", &p[i]);
    }
    clock_t start, end;
    start = clock();
    int result = matrixChain(p, n + 1);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Minimum number of multiplication = %d ", result);
    printf("\nRunning Time = %f seconds ", time_taken);
```

```
    return 0;
}
```

```
// 4) To Implement 0/1 knapsack problem using Dynamic Programming
approach for the user entered input and then find the running time of
implementation using time() function.

#include <stdio.h>
#include <time.h>

#include <stdio.h>
#include <time.h>

int max(int a, int b)
{
    if (a > b)
    {
        return a;
    }
    else
    {
        return b;
    }
}

int knapsack(int capacity, int weight[], int value[], int n)
{
    int dp[n + 1][capacity + 1];
    for (int i = 0; i <= n; i++)
    {
        for (int w = 0; w <= capacity; w++)
        {
            if (i == 0 || w == 0)
            {
                dp[i][w] = 0;
            }
            else if (weight[i - 1] <= w)
            {
                dp[i][w] = max(value[i-1] + dp[i-1][w-weight[i-1]],
dp[i-1][w]);
```

```
            }
            else
            {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    return dp[n][capacity];
}

int main()
{
    int n, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int value[n], weight[n];
    printf("\nEnter Values: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &value[i]);
    }
    printf("\nEnter Weights: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &weight[i]);
    }
    printf("\nEnter Knapsack Capacity: ");
    scanf("%d", &capacity);
    clock_t start, end;
    start = clock();
    int result = knapsack(capacity, weight, value, n);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nMaximum Profit = %d ", result);
    printf("\nRunning Time = %f seconds ", time_taken);
    return 0;
}
```

```c
// 5) To Implement Fractional knapsack using Greedy approach for the
user entered input and then find the running time of implementation
using time() function

#include <stdio.h>
#include <time.h>

int main()
{
    int n, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int value[n], weight[n], ratio[n];
    printf("Enter Value: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &value[i]);
    }
    printf("Enter Weight: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &weight[i]);
        ratio[i] = (float)value[i] / weight[i];
    }
    printf("Enter Capacity: ");
    scanf("%d", &capacity);
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (ratio[i] < ratio[j])
            {
                float tempRatio = ratio[i];
                ratio[i] = ratio[j];
                ratio[j] = tempRatio;
                int tempValue = value[i];
                value[i] = value[j];
                value[j] = tempValue;
                int tempWeight = weight[i];
                weight[i] = weight[j];
                weight[j] = tempWeight;
            }
        }
```

```
    }
    clock_t start, end;
    start = clock();
    float totalProfit = 0;
    for (int i = 0; i < n; i++)
    {
        if (weight[i] <= capacity)
        {
            totalProfit += value[i];
            capacity -= weight[i];
        }
        else
        {
            totalProfit += ratio[i] * capacity;
            break;
        }
    }
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Maximum Profit = %.2f ", totalProfit);
    printf("\nRunning Time = %f seconds", time_taken);
    return 0;
}
```

```
// 6) To Implement Huffman Coding using Greedy approach for the user
entered input and then find the running time of implementation using
time() function.

#include <stdio.h>
#include <time.h>
#include <stdlib.h>

struct Node
{
    char data;
    int freq;
    struct Node *left, *right;
};

struct Node *createNode(char data, int freq)
```

```
{
    struct Node *temp = (struct Node *)malloc(sizeof(struct Node));
    temp->data = data;
    temp->freq = freq;
    temp->left = temp->right = NULL;
    return temp;
}

void printCodes(struct Node *root, int arr[], int top)
{
    if (root->left)
    {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }
    if (root->right)
    {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }
    if (!root->left && !root->right)
    {
        printf("%c: ", root->data);
        for (int i = 0; i < top; i++)
        {
            printf("%d", arr[i]);
        }
        printf("\n");
    }
}

int main()
{
    int n;
    printf("Enter number of characters: ");
    scanf("%d", &n);
    char ch[n];
    int freq[n];
    printf("Enter Characters: ");
    for (int i = 0; i < n; i++)
    {
        scanf(" %c", &ch[i]);
    }
```

```
    printf("Enter Frequencies: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &freq[i]);
    }
    clock_t start, end;
    start = clock();
    struct Node *nodes[100];
    for (int i = 0; i < n; i++)
    {
        nodes[i] = createNode(ch[i], freq[i]);
    }
    int size = n;
    while (size > 1)
    {
        int min1 = 0, min2 = 1;
        if (nodes[min1]->freq > nodes[min2]->freq)
        {
            int t = min1;
            min1 = min2;
            min2 = t;
        }
        for (int i = 2; i < size; i++)
        {
            if (nodes[i]->freq < nodes[min1]->freq)
            {
                min2 = min1;
                min1 = i;
            }
            else if (nodes[i]->freq < nodes[min2]->freq)
            {
                min2 = i;
            }
        }
        struct Node *newNode = createNode('$', nodes[min1]->freq +
nodes[min2]->freq);
        newNode->left = nodes[min1];
        newNode->right = nodes[min2];
        nodes[min1] = newNode;
        nodes[min2] = nodes[size - 1];
        size--;
    }
    int arr[100], top = 0;
```

```
    printf("Huffman Coding: \n");
    printCodes(nodes[0], arr, top);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Running Time: %f seconds", time_taken);
    return 0;
}
```

```
// 7) To Implement single-source shortest path algorithm namely
Dijkstra algorithm for the user entered input and then find the running
time of implementation using time() function

#include <stdio.h>
#include <limits.h>
#include <time.h>
#define MAX 100

int minDistance(int dist[], int visited[], int n)
{
    int min = INT_MAX;
    int min_index;
    for (int i = 0; i < n; i++)
    {
        if (visited[i] == 0 && dist[i] < min)
        {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}

void Dijkstra(int graph[MAX][MAX], int n, int source)
{
    int dist[n];
    int visited[n];
    for (int i = 0; i < n; i++)
    {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }
```

```
    dist[source] = 0;
    for (int count = 0; count < n - 1; count++)
    {
        int u = minDistance(dist, visited, n);
        visited[u] = 1;
        for (int v = 0; v < n; v++)
        {
            if (!visited[v] && graph[u][v] && dist[u] != INT_MAX &&
dist[u] + graph[u][v] < dist[v])
            {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printf("\nVertex\tDistance from source\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d \t\t %d\n", i, dist[i]);
    }
}

int main()
{
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int graph[MAX][MAX];
    printf("Enter Adjancency Matrix: \n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
    int source;
    printf("Enter Source: ");
    scanf("%d", &source);
    clock_t start, end;
    start = clock();
    Dijkstra(graph, n, source);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
    printf("Running Time = %f seconds", time_taken);
    return 0;
}
// 8) To Implement single-source shortest path algorithm namely Bellman
Ford for the user entered input and then find the running time of
implementation using time() function.

#include <stdio.h>
#include <limits.h>
#include <time.h>

struct Edge
{
    int src, dest, weight;
};

void bellmanFord(struct Edge edges[], int V, int E, int source)
{
    int dist[V];
    for (int i = 0; i < V; i++)
    {
        dist[i] = INT_MAX;
    }
    dist[source] = 0;
    for (int i = 0; i < V - 1; i++)
    {
        for (int j = 0; j < E; j++)
        {
            int u = edges[j].src;
            int v = edges[j].dest;
            int w = edges[j].weight;
            if (dist[u] != INT_MAX && dist[u] + w < dist[v])
            {
                dist[v] = dist[u] + w;
            }
        }
    }

    for (int j = 0; j < E; j++)
    {
        int u = edges[j].src;
        int v = edges[j].dest;
        int w = edges[j].weight;
```

```
        if (dist[u] != INT_MAX && dist[u] + w < dist[v])
        {
            dist[v] = dist[u] + w;
        }
    }

    printf("\nVertex\t\tDistance from source\n");
    for (int i = 0; i < V; i++)
    {
        printf("%d \t\t %d\n", i, dist[i]);
    }
}

int main()
{
    int V, E;
    printf("Enter number of Vertices: ");
    scanf("%d", &V);
    printf("Enter number of Edges: ");
    scanf("%d", &E);
    struct Edge edges[E];
    printf("Enter Source, Destination, Weight: \n");
    for (int i = 0; i < E; i++)
    {
        scanf("%d%d%d", &edges[i].src, &edges[i].dest,
&edges[i].weight);
    }
    int source;
    printf("Enter Source: ");
    scanf("%d", &source);
    clock_t start, end;
    start = clock();
    bellmanFord(edges, V, E, source);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Running Time = %f seconds", time_taken);
    return 0;
}
```

```c
// 9) To Implement String Matching Algorithm namely Rabin-Karp
algorithm for the user entered input and then find the running time of
implementation using time() function

#include <stdio.h>
#include <string.h>
#include <time.h>

#define d 256

void rabinKarp(char text[], char pattern[], int q)
{
    int n = strlen(text);
    int m = strlen(pattern);
    int p = 0, t = 0, h = 1;
    for (int i = 0; i < m - 1; i++)
    {
        h = (h * d) % q;
    }
    for (int i = 0; i < m; i++)
    {
        p = (d * p + pattern[i]) % q;
        t = (d * t + text[i]) % q;
    }
    for (int i = 0; i <= n - m; i++)
    {
        if (p == t)
        {
            int j;
            for (j = 0; j < m; j++)
            {
                if (text[i + j] != pattern[j])
                {
                    break;
                }
            }
            if (j == m)
            {
                printf("Pattern found at index %d\n", i);
            }
        }
        if (i < n - m)
        {
```

```
            t = (d * (t - text[i] * h) + text[i + m]) % q;
            if (t < 0)
            {
                t = t + q;
            }
        }
    }
}

int main()
{
    char text[100];
    char pattern[100];
    printf("Enter String: ");
    scanf("%s", text);
    printf("Enter Pattern: ");
    scanf("%s", pattern);
    int q = 101;
    clock_t start, end;
    start = clock();
    rabinKarp(text, pattern, q);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Running Time = %f seconds", time_taken);
    return 0;
}
```

```
// 10) To Implement String Matching Algorithm namely KMP algorithm for
the user entered input and then find the running time of implementation
using time() function .

#include <stdio.h>
#include <string.h>
#include <time.h>

void computeLPS(char pattern[], int m, int LPS[])
{
    int len = 0;
    LPS[0] = 0;
    int i = 1;
    while (i < m)
```

```
    {
        if (pattern[i] == pattern[len])
        {
            len++;
            LPS[i] = len;
            i++;
        }
        else
        {
            if (len != 0)
            {
                len = LPS[len - 1];
            }
            else
            {
                LPS[i] = 0;
                i++;
            }
        }
    }
}

void KMP(char text[], char pattern[])
{
    int n = strlen(text);
    int m = strlen(pattern);
    int LPS[m];
    computeLPS(pattern, m, LPS);
    int i = 0, j = 0;
    while (i < n)
    {
        if (pattern[j] == text[i])
        {
            i++;
            j++;
        }
        if (j == m)
        {
            printf("Pattern found at index %d\n", i - j);
            j = LPS[j - 1];
        }
        else if (i < n && pattern[j] != text[i])
        {
```

```
            if (j != 0)
            {
                j = LPS[j - 1];
            }
            else
            {
                i++;
            }
        }
    }
}

int main()
{
    char text[100];
    char pattern[100];
    printf("Enter String: ");
    scanf("%s", text);
    printf("Enter Pattern: ");
    scanf("%s", pattern);
    clock_t start, end;
    start = clock();
    KMP(text, pattern);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Running Time = %f seconds", time_taken);
    return 0;
}
```

```
// 11) To Implement Max flow Algorithm namely ford fulkerson's
algorithm for the user entered input and then find the running time of
implementation using time() function.

#include <stdio.h>
#include <time.h>
#include <string.h>
#include <limits.h>
#define V 6

int bfs(int graph[V][V], int s, int t, int parent[])
{
```

```
    int visited[V];
    for (int i = 0; i < V; i++)
    {
        visited[i] = 0;
    }
    int queue[V];
    int front = 0, rear = 0;
    queue[rear++] = s;
    visited[s] = 1;
    parent[s] = -1;
    while (front < rear)
    {
        int u = queue[front++];
        for (int v = 0; v < V; v++)
        {
            if (visited[v] == 0 && graph[u][v] > 0)
            {
                queue[rear++] = v;
                parent[v] = u;
                visited[v] = 1;
            }
        }
    }
    return visited[t];
}

int fordFukerson(int graph[V][V], int source, int sink)
{
    int u, v;
    int residual[V][V];
    for (int u = 0; u < V; u++)
    {
        for (int v = 0; v < V; v++)
        {
            residual[u][v] = graph[u][v];
        }
    }
    int parent[V];
    int maxFlow = 0;
    while (bfs(residual, source, sink, parent))
    {
        int pathFlow = INT_MAX;
        for (v = sink; v != source; v = parent[v])
```

```
        {
            u = parent[v];
            if (residual[u][v] < pathFlow)
            {
                pathFlow = residual[u][v];
            }
        }
        for (v = sink; v != source; v = parent[v])
        {
            u = parent[v];
            residual[u][v] -= pathFlow;
            residual[v][u] += pathFlow;
        }
        maxFlow += pathFlow;
    }
    return maxFlow;
}

int main()
{
    int graph[V][V];
    printf("Enter 6*6 Matrix:\n");
    for (int i = 0; i < V; i++)
    {
        for (int j = 0; j < V; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
    int source, sink;
    printf("Enter Source: ");
    scanf("%d", &source);
    printf("Enter Sink: ");
    scanf("%d", &sink);
    clock_t start, end;
    start = clock();
    int maxFlow = fordFukerson(graph, source, sink);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Maximum Flow = %d\n", maxFlow);
    printf("Running Time = %f seconds", time_taken);
    return 0;
}
```

```
// 12) To Implement Max flow Algorithm namely Bipartite Matching
problem for the user entered input and then find the running time of
implementation using time() function

#include<stdio.h>
#include<string.h>
#include<time.h>
#define M 3
#define N 3

int bpm(int graph[M][N], int u, int seen[], int matchR[]) {
for(int v = 0; v < N; v++) {
if(graph[u][v] && seen[v] == 0) {
seen[v] = 1;
if(graph, matchR[v], seen, matchR) {
matchR[v] = u;
return 1;
}
}
}
return 0;
}

int maxBPM(int graph[M][N]) {
int matchR[N];
for(int i = 0; i < N; i++) {
matchR[i] = -1;
}
int result = 0;
for(int u = 0; u < M; u++) {
int seen[N];
for(int i = 0; i < N; i++) {
seen[i] = 0;
}
if(bpm(graph, u, seen, matchR)) {
result++;
}
}
return result;
```

```
}

int main() {
int graph[M][N];
printf("Enter 3*3 Matrix:\n");
for(int i = 0; i < M; i++) {
for(int j = 0; j < N; j++) {
scanf("%d",&graph[i][j]);
}
}
clock_t start, end;
start = clock();
int result = maxBPM(graph);
end = clock();
double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Maximum Matching = %d\n", result);
printf("Running Time = %f seconds", time_taken);
return 0;
}
```

```
// 13) To Implement Sum of subsets Problem using Backtracking Agorithm
for the user entered input and then find the  running time of
implementation using time() function.

#include <stdio.h>
#include <time.h>

int set[100];
int subset[100];
int n, target;
void sumOfSubset(int index, int currentSum)
{
    if (currentSum == target)
    {
        printf("{ ");
        for (int i = 0; i < index; i++)
        {
            if (subset[i] == 1)
            {
                printf("%d ", set[i]);
            }
        }
```

```
        printf("}\n");
        return;
    }
    if (currentSum > target || index == n)
    {
        return;
    }
    subset[index] = 1;
    sumOfSubset(index + 1, currentSum + set[index]);
    subset[index] = 0;
    sumOfSubset(index + 1, currentSum);
}
int main()
{
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &set[i]);
    }
    printf("Enter target sum: ");
    scanf("%d", &target);
    clock_t start, end;
    start = clock();
    printf("\nSubsets are:\n");
    sumOfSubset(0, 0);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Running Time = %f seconds", time_taken);
    return 0;
}
```

```
// 14) To Implement 8 Queens Problem using Backtracking Algorithm for
the user entered input and then find the running time of implementation
using time() function.

#include <stdio.h>
#include <time.h>

int board[20][20];
int N;
```

```
void printBoard()
{
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

int isSafe(int row, int col)
{
    int i, j;
    for (i = 0; i < col; i++)
    {
        if (board[row][i] == 1)
        {
            return 0;
        }
    }
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
    {
        if (board[i][j] == 1)
        {
            return 0;
        }
    }
    for (i = row, j = col; i < N && j >= 0; i++, j--)
    {
        if (board[i][j] == 1)
        {
            return 0;
        }
    }
    return 1;
}

int solveQueens(int col)
{
    if (col >= N)
    {
```

```
        return 1;
    }
    for (int i = 0; i < N; i++)
    {
        if (isSafe(i, col))
        {
            board[i][col] = 1;
            if (solveQueens(col + 1))
            {
                return 1;
            }
            board[i][col] = 0;
        }
    }
    return 0;
}

int main()
{
    printf("Enter number of Queens: ");
    scanf("%d", &N);
    clock_t start, end;
    start = clock();
    if (solveQueens(0))
    {
        printf("Solution Exists:\n");
        printBoard();
    }
    else
    {
        printf("Solution does not exists!");
    }
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Running Time = %f seconds", time_taken);
    return 0;
}
```

```c
// 15) To Implement 15 puzzle problem using Branch and Bound algorithm
for the user entered input and then find the running time of
implementation using time() function .

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 4

int goal[N][N] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12},
    {13, 14, 15, 0}};

void printBoard(int board[N][N])
{
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            printf("%2d ", board[i][j]);
        }
        printf("\n");
    }
}

int calculateCost(int board[N][N])
{
    int count = 0;
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (board[i][j] != 0 && board[i][j] != goal[i][j])
            {
                count++;
            }
        }
    }
    return count;
}
```

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

void solvePuzzle(int board[N][N], int x, int y)
{
    int cost = calculateCost(board);
    printf("\nInitial Board:\n");
    printBoard(board);
    printf("Cost = %d\n", cost);
    if (x + 1 < N)
    {
        swap(&board[x][y], &board[x + 1][y]);
        printf("Move Blank Down!\n");
        printBoard(board);
        printf("Cost = %d\n", calculateCost(board));
        swap(&board[x][y], &board[x + 1][y]);
    }
    if (y + 1 < N)
    {
        swap(&board[x][y], &board[x][y + 1]);
        printf("Move Blank Right!\n");
        printBoard(board);
        printf("Cost = %d", calculateCost(board));
        swap(&board[x][y], &board[x][y + 1]);
    }
}

int main()
{
    int board[N][N];
    printf("Enter 4*4 Matrix:\n");
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            scanf("%d", &board[i][j]);
        }
    }
    int x, y;
```

```
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (board[i][j] == 0)
            {
                x = i;
                y = j;
            }
        }
    }
    clock_t start, end;
    start = clock();
    solvePuzzle(board, x, y);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nRunning Time = %f seconds", time_taken);
    return 0;
}
```

```
// 16) To Implement Vertex Cover Problem
// using Approximation Algorithm

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define V 20
int main()
{
    int graph[V][V];
    srand(time(0));
    // Generate random graph
    for(int i = 0; i < V; i++)
    {
        for(int j = 0; j < V; j++)
        {
            if(i == j)
            {
                graph[i][j] = 0;
            }
            else
            {
```

```
                graph[i][j] = rand() % 2;
            }
        }
    }
    int visited[V];
    for(int i = 0; i < V; i++)
    {
        visited[i] = 0;
    }
    clock_t start, end;
    start = clock();
    printf("Edges in Graph:\n");
    for(int i = 0; i < V; i++)
    {
        for(int j = i + 1; j < V; j++)
        {
            if(graph[i][j] == 1)
            {
                printf("(%d,%d) ", i, j);
            }
        }
    }
    printf("\n\nApproximate Vertex Cover:\n");
    // Approximation Algorithm
    for(int u = 0; u < V; u++)
    {
        if(visited[u] == 0)
        {
            for(int v = 0; v < V; v++)
            {
                if(graph[u][v] == 1 &&
                   visited[v] == 0)
                {
                    visited[u] = 1;
                    visited[v] = 1;
                    printf("%d %d ",
                           u, v);
                    break;
                }
            }
        }
    }
    end = clock();
```

```
    double time_taken =
    ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\n\nRunning Time = %f seconds\n",
           time_taken);
    return 0;
}
```